

Firewall, Intrusion Detection, and INT using P4

Objectives

The objectives of this project are:

- Familiarize yourself with the P4 language ecosystem,
- Explore control plane APIs,
- Implement a simple L2 firewall using P4,
- Implement a simple intrusion detection system (IDS) using P4,
- Implement a simple In-band Network Telemetry (INT) using P4, and
- Experiment with AI coding.

AI Policy

We recommend that you do not use any AI tools for Tasks I–III. These tasks build foundational skills in P4 programming. You may use AI in Task IV. In any case, you are required to understand every single line of your code—we will conduct oral check-ins. If you used AI in any task, you are required to submit a separate document named **ai_log.pdf (≤ 5 pages)**. It should describe:

1. Which specific tasks/steps you used AI assistance for.
2. The most useful prompts and brief summaries of what they produced.
3. One case where the AI output was wrong, incomplete, or unhelpful, and what you did instead.

If you did not use any AI tools, write “No AI tools used” in ai_log.pdf. This will not affect your grade.

Introduction

Programmable network switches have been gaining popularity over the last few years. Unlike fixed-function switches, programmable switches enable operators to quickly realize and compose various networking functions in the data plane. Examples of these functions include packet forwarding, firewall, network address translation (NAT), traffic engineering, load balancing, and intrusion detection systems (IDS). Programmable switches accelerate innovations in ISP, enterprise, and datacenter networks by enabling the rapid development and testing of new protocols and ideas. P4 is the language used to program network switches.

In this project, you will first explore and experiment with the P4 environment. Second, you will implement a simple layer 2 (L2) forwarding logic using P4 and extend it to support firewalling, i.e., blocking some traffic flows. Third, you will implement a simple intrusion detection system (IDS) that looks for signatures of viruses and worms in packet payloads and stops traffic flows that contain these signatures. Finally, you will implement in-band network telemetry (INT).

Setup the Environment

Follow the  `setupDevelopmentEnvironment` document.

I. Explore the P4 Environment

Download and extract [simple-switch.zip](#). Work inside `simple-switch/program1`, which is a working L2 switch. The `simple-switch/program1` folder contains two files (`network.py` and `controller.py`) and a sub-folder (`p4src`). `network.py` is a Python script that sets up the environment and creates a simple network topology with one switch and four hosts directly connected to it. `controller.py` is another script that runs the control plane program to populate the table entries on the switch. These Python scripts use the open-source [p4-utils](#) library, which is already installed in the VM. The `p4src` sub-folder contains a file called `program.p4`, which is the P4 code that runs on the programmable switch.

I.1: Experiment with the L2 Forwarding Switch

First, create the network topology and start the `mininet` environment by executing the following while in the `simple-switch/program1/` folder:

```
sudo python network.py
```

This will show the `mininet` prompt, from which you can issue commands to interact with different elements of the network and send/receive traffic. Notice that running `network.py` will compile the P4 code and instantiate the software switch without further configurations. In this example, the switch architecture used is `v1model`, and the software switch is `bmw2`, both provided by the P4 community.

Include your answers and screenshots for all questions below in ONE PDF file and submit this file.

(1) Collect Basic Outputs and Traces

(1.a) Run the following commands in `mininet` and include screenshots of their outputs in your report.

a. `h2 ping h3 -c1`

- b. `pingall`
- c. `h1 traceroute h2`

(1.b) Exit `mininet` (issue `quit`) and modify the `network.py` to enable dumping PCAP (Packet Capture) files. PCAP files contain information about packets captured as they pass through the network. More information about PCAP and its format is available on the [tcpdump](http://tcpdump.org) website. To enable dumping PCAP, replace `disablePcapDumpAll` with `enablePcapDumpAll` in `network.py`.

Run the network again (`sudo python network.py`), then run the command `pingall`. Exit `mininet` and look at the generated PCAP files in the `./pcap` directory:

- Explain the meaning of `s1-eth1_in.pcap` and `s1-eth1_out.pcap`.
- Inspect the two files using the tool `tshark`, and include two screenshots of `tshark` outputs. (The `-r` option allows you to read from a capture file: `tshark -r filename`)

Note: Remember to disable PCAP file dumping after you finish this part.

(1.c) Exit `mininet` and modify the `network.py` to enable logging the packet processing pipeline. To enable logging, replace `disableLogAll` with `enableLogAll`. Run the network again, and call the command: `h1 ping h2 -c1`. Exit `mininet`. Inspect the generated log file in the `./log` directory using any text editor such as `vim`:

- Include a screenshot of the part of the log file (not the entire log file) showing the processing steps of the ping request packet.
- How many bytes were parsed? Why?
- What is the packet size? Why?

Note: Remember to disable logging after you are done with this part.

(2) Understand the Control Plane Program

Recall that the control plane populates the table with proper entries. This can be achieved using Python APIs or a command file (a text file containing commands to be executed by the control plane).

(2.a) In `controller.py`, implement the `dump_table()` function by calling `self.controller.table_dump`, which takes a table name as input. This function prints the table entries to the screen; its documentation is available [here](#).

Run the network and include a screenshot of the `table_dump` outputs in your report, which are printed before the `mininet` prompt and start with something like:

```
=====
TABLE ENTRIES
*****
Dumping entry 0x0
Match Key:
...
```

(2.b) Open the provided `s1-commands.txt` and explain the format of this file.

(2.c) In `network.py`, enable the control plane to use the command file by uncommenting the proper line. Ensure that `network.py` now uses the main script in `controller.py` along with the command file to populate the table with the same entries.

- Run the network and show a screenshot of the table output. The outputs will be printed before the mininet prompt.
- Adding duplicate forwarding rules to a data plane table is not allowed by default. However, when `controller.py` is run after `s1-commands.txt` commands, the switch overrides entries from the script source. Explain why this happens. Identify the function in `controller.py` that causes this effect.

I.2: Understand the Packet Processing Pipeline

You will explore the packet-processing steps within the switch by modifying the data-plane and control-plane programs. You need to enable dumping PCAP and/or logging.

(1) Modify the P4 Program

In `program.p4`, comment the line `dmac.apply()`. Run the network.

- Would this result in a compilation error? Explain.
- Execute a simple command such as `h1 ping h2 -c 1`. What are the outputs?
- Explain the outputs of the ping command. *Hint*: examine the log file.
- Is the `dmac` table included among the artifacts produced? Why? Show a screenshot of running the network to verify your answer.

Note: Remember to uncomment the line `dmac.apply()` before proceeding.

(2) Modify the Control Plane Program

In the `controller.py` program, comment the line that inserts an entry for the MAC address `00:00:0a:00:00:01`. Run the network.

- Show a screenshot of executing `pingall`.
- How many packets did `h1` send and receive? *Hint*: inspect the PCAP files.
- Explain what happened. *Hint*: inspect the log file.

II. Implement a Simple L2 Firewall

To forward a packet, the L2 switch needs to know the mapping between a destination MAC address and an outgoing port. Real L2 switches run a simple learning mechanism to dynamically create this mapping. We relax this requirement in this project and assume that the mapping is *static*.

We would like the given L2 switch to implement a simple firewall function in addition to the basic forwarding function. Specifically, the switch is required to drop packets transmitted from specific hosts. You need to write a P4 program, similar to `program1`, to implement these requirements.

Steps

- Copy the `program1` folder to a new folder named `firewall`.
- Implement the blocking logic in `program.p4`.
- Implement the control plane program in `controller.py`. The switch should drop all traffic from host `h1` to other hosts, while forwarding traffic from all other hosts, including traffic to `h1`. Essentially, this should be a one-way firewall.
- In your final program, run the following commands:
 - `pingall`. Show a screenshot of the outputs.
 - `h1 ping h2 -c 1`. Inspect the log file and provide a screenshot of the steps that led to the packet being dropped.

Hint: Think of the following potential design choices (you do **not** need to include answers to these questions in the report):

- Would you modify the existing `dmac` table or add a new one?
- What are the tradeoffs of both approaches?

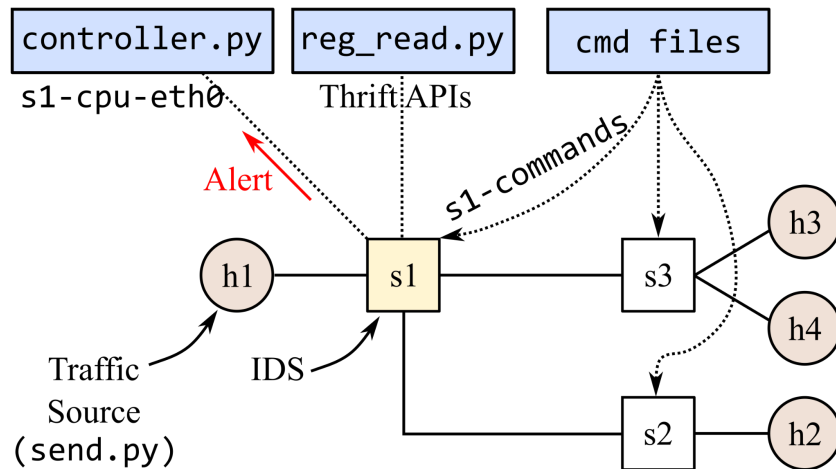
III. Implement a Simple IDS

An IDS detects malicious traffic, e.g., traffic carrying viruses and worms, and it alerts the administrator to potential threats. IDS systems can be broadly classified into anomaly-based and signature-based systems. The former attempts to infer the common behavior of normal traffic and flags suspicious traffic that deviates from it. The latter matches parts of the traffic against known malware signatures (or fingerprints). The traffic is identified as malicious if a signature is found.

In this part, you will implement a layer 4 (L4) signature-based IDS using P4, where the signature exists in the first few bytes in the TCP payload. In particular, you will implement two variants. The first one is *stateless* because it makes a decision based on the payload of individual TCP packets. The second variant is *stateful* because it inspects the payloads of multiple TCP packets within the same flow.

III. 1 Network Setup

The network topology consists of three switches and four hosts as shown in the following figure. All switches implement IPv4 forwarding logic. Switch `s1` should implement the IDS function to protect the network from malicious traffic. The control plane has three components: (i) `controller.py`: receives and prints intrusion alerts from the data plane, (ii) `reg_read.py`: uses Thrift APIs to read the number of dropped packets per flow, and (iii) the command text files: populate corresponding tables in the data plane.



III.2 IDS Interface and Operation

Both stateless and stateful IDS systems should be able to match *multiple* signatures across *different* traffic flows. A traffic flow refers to all packets belonging to one session between two end hosts. A flow is identified by five fields: source IP address, destination IP address, source port, destination port, and protocol.

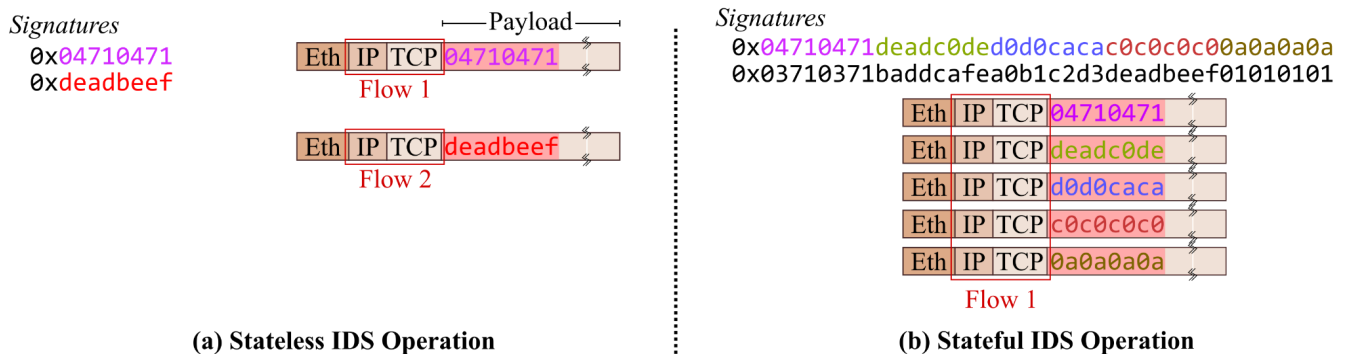
You should use tables to store signatures. Regardless of how you will implement the IDS, it should have at least the following two P4 constructs:

- The `signatures` table: the key is the known signature, and the action is `signature_hit`, which takes the CPU port number as an argument (CPU port 4 in this topology).

- The **counters** register array: an array of registers that count the total number of dropped packets per flow. Each item in the array has an index in the range of $[0, \text{FLOW_ENTRIES} - 1]$, and its value is an unsigned 32-bit integer. In this project, we set **FLOW_ENTRIES** to 4,096.

Note: when grading a submission, we will read and/or write different values to these P4 constructs using **our** scripts.

The following figure shows the difference between the stateless and stateful IDS systems. In the stateless IDS, the data plane inspects a constant number of bits at the beginning of the TCP payload; we refer to this constant number of bits as **PATTERN_WIDTH**. The IDS matches the first **PATTERN_WIDTH** bits in each packet against the known signatures. If a packet contains a malicious signature, it will be dropped, and the flow it belongs to should be blocked. That is, future packets in the same flow will be dropped. In the example of Figure (a) below, each of Flow 1 and Flow 2 matches a different known malicious signature.



In some cases, the signature may be distributed among multiple packets, as shown in Figure (b) above. The *stateful* IDS thus operates on **FLOW_PKT_THRESHOLD** *consecutive* packets per flow. For each packet, the data plane constructs a pattern by concatenating the first **PATTERN_WIDTH** bits of the TCP payload. The flow is blocked when the data plane matches the **FLOW_PKT_THRESHOLD** * **PATTERN_WIDTH** bits against a known signature. Notice that the **FLOW_PKT_THRESHOLD**th packet and all following packets in the flow should be dropped. The stateful IDS becomes stateless when **FLOW_PKT_THRESHOLD** is set to 1.

In other words, you should implement rolling concatenation to maintain a sliding window of the last **FLOW_PKT_THRESHOLD** packets. Each incoming packet should be appended to this buffer, and the complete pattern should be checked for malicious signatures.

Do not check for scenarios with fewer than `FLOW_PKT_THRESHOLD` packets to simplify implementation, as handling such cases would require maintaining an additional per-flow state to count packets.

Note: In the figure, we set `PATTERN_WIDTH` to 32 bits and `FLOW_PKT_THRESHOLD` to 5 consecutive packets for illustration only. Your implementation should be parametrized by `PATTERN_WIDTH` and `FLOW_PKT_THRESHOLD`.

Provided Code Skeleton

[Download the provided skeleton code here.](#)

The skeleton code for stateless and stateful IDS can be found in `ids_stateless` and `ids_stateful` directories, respectively. Each of these folders contains four Python files (`network.py`, `controller.py`, `reg_read.py`, and `send.py`), three command text files, and a sub-folder (`p4src`). The `network.py` sets up the environment and creates the network topology. The `controller.py` is another script that receives intrusion alerts from the data plane. The `reg_read.py` retrieves register values from the data plane. The `send.py` transmits packets to test your implementation. The `p4src` sub-folder contains a file called `program.p4`, which is the P4 code that runs on the programmable switch.

Note: We will run **various** scripts to grade your submission, so you are encouraged to rigorously test your code across a range of cases, especially extreme and boundary cases.

The workflow to run the provided scripts is:

1. Create the network topology and start the `mininet` environment by executing: `sudo python network.py`
Note: This script uses the `*-commands.txt` files to populate tables in the switches.
2. Run `controller.py` in another terminal: `sudo python controller.py`.
3. Transmit traffic from the `mininet` terminal: `mininet> h1 python send.py 10.0.2.2`.
4. Run `reg_read.py` in another terminal: `sudo python reg_read.py`.
5. Inspect the `controller.py` and `reg_read.py` terminals.

III.2 Implementing the Stateless IDS

The provided code in `ids_stateless/p4src/program.p4` contains multiple `// TODO` statements for you to implement. Although you may need to add P4 constructs, your implementation should use the `signatures` table and the `counters` register array.

The suggested steps to implement the stateless IDS in the `IDS_Ingress.apply()` action are:

1. Execute the action in the `flows` table to get the flow status (i.e., whether it is blocked).
Hint: You need to add a new P4 structure to hold this value.
2. If the flow is blocked, drop the packet and increment the corresponding item in `counters`.
3. If the flow is not blocked:
 - a. Check the `signatures` table
 - b. If there is a hit:
 - i. Send the packet to the CPU
 - ii. Increment the corresponding item in `counters`
 - iii. Maintain the information that the flow is blocked
 - c. If there is no hit, perform IPv4 forwarding.

III.3 Implementing the Stateful IDS

Similar to the stateless IDS case, the provided code in `ids_stateful/p4src/program.p4` contains multiple `// TODO` statements to be completed. You will also need to use the `signatures` table and the `counters` register array. (See the Appendix at the end for hints on tables, actions, and flows).

The suggested steps to implement the stateful IDS in the `IDS_Ingress.apply()` action are:

1. Execute the action in the `flows` table to get the flow status (i.e., whether it is blocked).
2. If the flow is blocked, drop the packet and increment the corresponding item in `counters`.
3. If the flow is not blocked:
 - a. Execute the action in the `build_pattern` table. This action should build the pattern by concatenating the first `PATTERN_WIDTH` bits of the TCP payload to the pattern maintained from previous packets.
Hint: You need to keep the concatenated bytes across multiple packets.
 - b. Check the `signatures` table
 - c. If there is a hit:
 - i. Send the packet to the CPU
 - ii. Increment the corresponding item in `counters`
 - iii. Maintain the information that the flow is blocked
 - d. If there is no hit, perform IPv4 forwarding.

IV. Implement In-band Network Telemetry (INT)

In-band Network Telemetry (INT) lets each switch a packet passes through stamp its internal state directly into the packet. The state may include, for example, the switch queue depth or delay experienced by packets before being forwarded. This gives operators visibility into congestion at the

exact switch and the moment it happened, rather than inferring it indirectly from round-trip time. INT is widely used in datacenter and AI-cluster networks to detect incast and hot spots in real time.

In this part, you will extend your P4 pipeline so that a switch recognizes specially tagged packets and stamps a custom telemetry header onto them as they pass through, and you will implement a Python collector that reads this telemetry back out on the receiving host.

Steps

- Use the provided [simple-INT.zip](#) starter code. After extracting it, work inside `simple-INT/int`. This starter uses an IPv4-forwarding pipeline rather than the L2 switch in Part I, because INT packets in this task are formatted as Ethernet -> int_t -> IPv4 -> payload.

The provided `network.py`, `controller.py`, and `send_int.py` are support files. You should not need to modify them for the basic task. Your main implementation work is in `p4src/program.p4` and `collect_int.py`.

In `p4src/program.p4`, define a custom header, `int_t`, with three fields: `hop_count` (8 bits), `queue_depth` (32 bits), and `hop_latency` (32 bits), for a total of 9 bytes in that order.

Add the INT header instance to the headers struct. Reserve `EtherType 0x1234` as the INT trigger. A packet carrying this `EtherType` is laid out as: Ethernet -> int_t -> IPv4 -> payload.

- Add a parser state that extracts `int_t` immediately after Ethernet when `etherType == 0x1234`, then transitions into IPv4 parsing.
- In the egress control block, stamp any packet where the INT header is valid: increment `hop_count` by 1, and copy the switch's own `standard_metadata.enq_qdepth` and `standard_metadata.deq_timedelta` fields into `queue_depth` and `hop_latency` respectively. Regular (non-INT) IPv4 packets must pass through untouched.
- In the deparser, emit Ethernet -> INT -> IPv4 for INT packets.
- In `collect_int.py`, complete the parser so it skips the 14-byte Ethernet header, parses the 9-byte INT header in network byte order, and prints `hop_count`, `queue_depth`, and `hop_latency` for each INT packet received. *Hint:* The 9-byte INT header corresponds to the Python struct format string "`!BII`": network byte order, then `hop_count` as 1 byte, `queue_depth` as 4 bytes, and `hop_latency` as 4 bytes.
- The provided controller configures the BMv2 egress queue toward h2, so queue depth is observable. Send a burst of INT-tagged packets and include a screenshot of `collect_int.py`

output showing `hop_count`, `queue_depth`, and `hop_latency`. At least some packets should show nonzero `queue_depth`.

- A tested workflow for this part is:
 - Terminal 1: Inside the Mininet `cd simple-INT/int; sudo python network.py`
 - prompt: `h2 sh -c 'python collect_int.py --count 20 > /tmp/int_output.txt &'`
 - Inside the Mininet prompt: `h1 python send_int.py --count 20 --interval 0`
 - Inside the Mininet prompt: `h2 cat /tmp/int_output.txt`

Hint: this task does not need the `flows/counters` machinery from Part III — there is no per-flow state to track, only per-packet stamping. The keyless-table pattern in the Appendix is still useful here if you want to invoke your stamping action from a table rather than inline in `apply`.

Suggested Implementation Methodology (AI-assisted)

You may use an AI coding assistant for this part; INT is a good task for it because the pattern is well-documented but fiddly to get exactly right (field widths, byte order, and BMv2's intrinsic metadata names are easy to mismatch). A productive way to work with an AI assistant is:

1. **Ask it to explain before you ask it to generate.** Start with a conceptual prompt, e.g., "explain how `standard_metadata.enq_qdepth` and `deq_timedelta` work in the BMv2 v1model architecture, and when in the pipeline they're populated". So you understand *why* the stamping has to happen in egress, not ingress, before you write any code.
2. **Generate the header and parser state first, in isolation.** Prompt for just the `int_t` header definition and the new parser state, then compile with `p4c` before moving on. This keeps you from debugging three new pieces of code at once.
3. **Generate the egress stamping logic separately**, and explicitly ask the assistant to show you the exact `standard_metadata` field names and widths it's using. Then cross-check those against the v1model reference in the Appendix/Resources section yourself. AI assistants frequently invent plausible-but-wrong field names for BMv2 intrinsic metadata.
4. **Write `collect_int.py`'s parser by hand—verifying the byte offsets.** Trace through 14 bytes of Ethernet header plus 9 bytes of your INT header yourself and confirm the offsets match before trusting the output.
5. **Use the AI to help you construct a test/debugging plan**, not just code. For example, ask it what each observed INT field should mean, how to sanity-check byte offsets, and why queue depth is measured from the BMv2 egress queue rather than from Linux traffic shaping.

Explain in your report where the AI assistant helped, and one point where its suggestion was wrong and you had to correct it.

Submission

You are required to submit:

1. You are required to structure your solution in ONE main folder that contains all code, called **p4_prj**:
 p4_prj/
 simple-switch/
 program1/
 firewall/
 simple-IDS/
 ids_stateless/
 ids_stateful/
 simple-INT/
 int/
 p4_report.pdf
 ai_log.pdf # only if AI was used
2. Compress the entire **p4_prj** into **p4_prj.zip**. Your code should compile and run without any errors. If the code does not compile, it will receive zero credits.
3. Include one report with all your answers and screenshots, named **p4_report.pdf**, in the zip file. If you used AI for any task, include **ai_log.pdf** as described above.

Appendix: Helpful P4 Primitives and Constructs

Running Actions without Matching Header Fields

You may need to run an action without matching any header fields. To achieve this, you can implement a table with a single entry and a default action set to the required action to be executed:

```
table MyTable {  
    actions = { my_action; }  
    size = 1;  
    default_action = my_action();  
}
```

You can invoke the action by calling: `MyTable.apply()`

Identifying a Flow

Register arrays require the index to be an integer value. To maintain information (e.g., a counter) about a flow, you need to map some fields in the packet to a single integer value. In this project, we will apply a hash function to the five-tuple of an IP packet using the CRC32 hashing algorithm.

The hash function has the following interface:

```
void hash<O, T, D, M>(out O result, in HashAlgorithm algo, in T base, in D data, in M max);
```

- **result**: is the variable holding the calculated flow index (i.e., hash)
- **algo**: should be set to `HashAlgorithm.crc32`
- **base**: should be set to `(bit<16>)0`
- **data**: the five-tuple fields of the IP packet. To pack multiple fields:
`{field_1, field_2, field_3}`
- **max**: maximum value used in the hash function. Should be set to the maximum number of supported flows by the IDS (`FLOW_ENTRIES`).

Check the `v1model` definition file for additional documentation.

Checking a Table Hit or Miss

P4 allows checking whether applying a table resulted in a hit or miss. This can be useful to run a different logic according to the result of a previous table. To perform this conditional execution:

```
if(MyTable.apply().miss) {  
    MissTable.apply();  
} else {  
    HitTable.apply();  
}
```

Resources

- P4_16 language specifications: <https://p4.org/p4-spec/docs/P4-16-working-spec.html>
- BMV2: <https://github.com/p4lang/behavioral-model>
- v1model: https://github.com/p4lang/behavioral-model/blob/main/docs/simple_switch.md
- P4 Cheat Sheet: <https://opennetworking.org/wp-content/uploads/2020/12/p4-cheat-sheet.pdf>